

seL4 boot process

Page Content

- [Overview](#)
- [RasPi](#)
- [zynq7000](#)
- [i.MX6](#)
- [RISC-V](#)

Overview

- on many system [U-Boot](#) is the boot loader started by the ROM code.
- U-Boot then starts the [seL4 elfloader](#), which loads the kernel and user task
- ⚠ many seL4 drivers that not fully working stand-alone, they seem to relay on some system setup done by U-Boot

RasPi

- see also [TRENTOS-M on RPi 3](#)
- see also [Raspberry Pi](#)

zynq7000

- [U-Boot](#) loads elfloader
- [TRENTOS-M on Xilinx Zynq-7000](#)

i.MX6

- [U-Boot](#) loads elfloader
- [TRENTOS-M on NXP i.MX 6](#)

RISC-V

- bootloader image contains
 - proxy kernel
 - DTB
 - seL4 kernel
 - seL4 user tasks (at least the root task, maybe more)
- proxy kernel (BBL) starts in M-Mode
 - basic core initialization
 - reads DTB
 - initialized CLINT, PLIC
 - installs a SBI handler (see [RISC-V BBL \(Proxy Kernel\)](#))
 - installs emulation for mtime register access
- ELF Loader runs and loads seL4 image and root task image into memory
 - parameters
 - r0: RISC-V specific: hart ID
 - **ToDo:**
 - we can drop this step if we have everything ready-to-run in ROM already
 - clarify details and how RISC-V and ARM ports different here
 - places some meta data from the loaded user elf image right after it in physical memory
 - function `load_elf()` in `elfloader-tool/src/common.c`, see parameter "keep_headers" of
 - seL4 treats it as an extra 4k part of the user image when creating address space objects
 - can be dropped if root task does not use this.
- kernel entry at symbol `_start` in `src/arch/riscv/head.S`
 - parameters
 - a0 (x10): seL4 generic: phys addr where user image starts in ROM
 - a1 (x11): seL4 generic: phys addr where user image ends in ROM
 - a2 (x12): seL4 generic: offset between phys and virt addresses in image, to be used as: `virt = phys - offs`
 - a3 (x13): seL4 generic: virtual entry point of user image
 - before <https://github.com/seL4/seL4/commit/3914931b32e19c38efc169ae18a2961453bad243> there were also these parameters:
 - a4 (x14): RISC-V specific: hart ID
 - a5 (x15): RISC-V specific: phys address of DTB in ROM
- RISC-V specific:
 - setup sp (currently there is no multi-core support)
 - initialize gp register

- initialize sscratch with x0 (=0)
- init_kernel() from src/arch/riscv/kernel/boot.c
 - try_init_kernel() from src/arch/riscv/kernel/boot.c
 - init_cpu() from src/arch/riscv/kernel/boot.c
 - activate_kernel_vspace() in include/arch/riscv/arch/machine.h
 - activates MMU - (was already activated by the elf-loader)
 - init_plat(dtb) in src/arch/riscv/kernel/boot.c
 - older versions had DTB parameter, this is obsolete now
 - parseFDT((void*)dtb.start)
 - looks for "device_type" and "memory". Device Tree documentation says

The memory layout is described by the following node:

```
{
    #address-cells = <(n)>;
    #size-cells = <(m)>;
    memory {
        device_type = "memory";
        reg = <(baseaddr1) (size1)
              (baseaddr2) (size2)
              ...
              (baseaddrN) (sizeN)>;
    };
    ...
};
```

A memory node follows the typical device tree rules for "reg" property:

n:	number of cells used to store base address value
m:	number of cells used to store size value
baseaddrX:	defines a base address of the defined memory bank
sizeX:	the size of the defined memory bank

- init_freemem(userImage, dtb) in src/arch/riscv/kernel/boot.c
 - older versions had DTB parameter, this is obsolete no. DTB contents is unused, only the physical address of the end of DTB is used. A code comment says:

```
// We ignore all physical memory before the dtb as the current riscv-pk (proxy
kernel)
// that we use for loading is broken and provides an incorrect memory map where
// it claims that the memory that is used to provide the m-mode services are
// free physical memory. As there is no interface to determine what the memory
// reserved for this is we simply hope it placed the dtb after itself and
exclude
// all memory up until then.
```

- create_it_address_space() from src/arch/riscv/kernel/vspace.c
 - create address space of initial thread
- create_frames_of_region() from src/kernel/boot.c
 - create_mapped_it_frame_cap() from seL4/src/arch/riscv/kernel/boot.c (ToDo: shouldn't this be in src/arch/riscv/kernel/vspace.c ?)
 - **ToDo:** seems parameter "executable" is not used in this function ?
 - map_it_frame_cap() in src/arch/riscv/kernel/vspace.c
 - ⚠ all page table entries are created as user/RWX
 - create_idle_thread() from src/kernel/boot.c
 - create_initial_thread(...., v_entry) from src/kernel/boot.c
 - schedule() in src/kernel/thread.c
 - activateThread() in src/kernel/thread.c
- calls restore_user_context()